Software Test Plan (STP) – Bank Management System

Project: Bank Management System
Version: 1.0
Authors: Kruthik Nandan, Himesh Raghav, Janhavi Ramakrishnan
Date: 13-11-2026
Status: Draft

1. Introduction

Purpose: This Software Test Plan defines the scope, approach, resources, responsibilities, and evidence required to verify the Bank Management System against its SRS (Version 2.0). It is intended to guide unit, integration, system, security, performance, usability, and acceptance testing.

Scope: Testing covers authentication and session handling; Customer, Employee, and Admin permissions; account and balance management; deposits and withdrawals; transaction recording; internal transfers and rollback; loan application and review; transaction history, filtering, pagination and CSV export; notifications; beneficiaries; profile settings; security controls; audit logging; health/readiness checks; and SQLite backup/recovery.

References: Bank Management System SRS v2.0 (12-11-2026); repository README and source tree at https://github.com/KN15-git/Bank_Management_5D; backend pytest suite and Postman collection where present.

Definitions: BMS (Bank Management System), FR (Functional Requirement), NFR (Non-functional Requirement), SR (Security Requirement), RBAC (Role-Based Access Control), CSRF (Cross-Site Request Forgery), RTM (Requirements Traceability Matrix), UAT (User Acceptance Testing), paise (1/100 of an Indian rupee).

2. Test Items

- Flask REST API and backend business logic (Python 3.10+, Flask, Flask-Login, bcrypt).
- SQLite database schema, constraints, transactions, audit records, and backup/recovery functions.
- React 18 frontend, React Router v6 pages/components, and Vite development build.
- Authentication, session, CSRF, role enforcement, account, deposit, withdrawal, transfer, and loan workflows.
- Transaction history, filters, pagination, CSV export, notifications, beneficiaries, and profile settings.
- Health/readiness endpoints, response timing headers, security/cache headers, and

administrative functions.
- Existing pytest tests and Postman API collection (where maintained in the repository).

3. Features to be Tested

The following requirement identifiers are taken from SRS v2.0. Test cases are specified in Annex A.
- FR-002 to FR-009: Authentication, RBAC, account/balance, deposits, withdrawals, transaction recording, transfers, and atomic rollback.
- FR-010 to FR-012: Loan application, authorized review, approval/rejection, status changes, and rejection reasons.
- FR-013 to FR-015: Transaction history, filtering/sorting/pagination, and safe CSV statement export.
- FR-016 to FR-018: Notifications, beneficiary management, and profile/account settings.
- NFR-019 to NFR-024: Data security/role access, response time, availability/reliability, audit logging, UI/accessibility, and backup/recovery.
- SR-001 to SR-005 and SO-01/SO-02: Authentication security, CSRF, authorization and data isolation, security/cache headers, auditability, confidentiality, and integrity.

4. Features Not to be Tested

- External core-banking, payment-gateway, inter-bank settlement, SMS, email, or third-party messaging integrations; these are not specified as implemented interfaces in the supplied SRS.
- Physical banking hardware, Bank Management System terminals, card readers, cash dispensers, passbooks, and printer firmware; the project is a browser-based application.
- Production infrastructure, cloud failover, disaster-recovery site availability, and production TLS configuration unless a deployment environment is separately supplied.
- Regulatory certification or formal compliance audits (for example, PCI-DSS); no such certification scope is specified in the project SRS.
- Real customer data or live financial transactions. All tests must use synthetic accounts and data.

5. Test Approach / Strategy

Levels:
- Unit testing: Validate individual backend functions, validators, permission checks, and utility functions.
- Integration testing: Verify Flask routes, SQLite persistence, transaction boundaries, authentication/session behaviour, and frontend-to-API contracts.
- System testing: Execute complete role-based workflows through the React UI and REST API.
- Acceptance testing: Validate SRS acceptance criteria with representative Customer, Employee, and Admin workflows.

Types:
- Functional, negative, boundary, regression, database-integrity, security, API,

usability/accessibility, performance, reliability, and backup/restore testing.
- Exploratory testing for role-specific navigation, error messages, and unusual workflow sequences.

Entry Criteria: A versioned build is available; dependencies install successfully; a disposable test database can be created; seed/test data is available; the backend and frontend start; and test accounts for all three roles are provisioned.

Exit Criteria: All planned high-priority cases have been executed; no open Critical/High severity defect blocks a banking or security invariant; all failed cases have defect records; the RTM is updated with evidence; and unresolved gaps/risks are explicitly accepted by the project owner. Do not report a case as passed without execution evidence.

5.1 Security Validation

- Verify signed HttpOnly session behaviour and bcrypt password hashing; ensure passwords are never stored or returned in plaintext (SR-001).
- Verify state-changing requests reject missing, invalid, or mismatched X-CSRF-Token values (SR-002).
- Verify server-side role checks and customer ownership isolation for direct API requests, not only hidden UI controls (SR-003).
- Verify required HTTP security headers and cache-protection headers on relevant responses (SR-004); compare against the project's security configuration rather than assuming a particular deployment TLS setup.
- Verify audit events capture the required actor/action/time/details and that unauthorized users cannot read, export, or alter audit data (SR-005).
- Exercise login lockout after five failed attempts for the configured 15-minute period; test session expiry/logout and access to protected routes.
- Attempt parameter tampering, invalid IDs, negative/zero/oversized amounts, self-transfer, cross-customer access, CSV formula payloads, and unauthorized administrative operations.

6. Test Environment

Hardware: Developer/QA workstation capable of running the browser, Python backend, and Node.js frontend; no Bank Management System or banking hardware is required.
Software: Python 3.10+; Flask REST API; SQLite with foreign-key enforcement; React 18; React Router v6; Vite; supported desktop browser; pytest; and the repository's Postman collection. Install dependencies from backend requirements and frontend package manifest.
Tools: pytest for backend automation; Postman for API checks; browser developer tools for UI/network/session inspection; SQLite inspection tools for data-integrity verification; and a spreadsheet application for CSV export checks. Use the repository's existing test tooling before adding new dependencies.
Test Data: Synthetic customers, employee and admin accounts; multiple active/frozen/closed accounts; accounts with zero, low, and high balances; valid/invalid beneficiaries; loan applications in each status; and transaction histories spanning date

boundaries. Never use real credentials or live customer data.
Isolation: Use a separate disposable test database. Backup/restore tests must operate on copies and must not overwrite a developer's working database.

7. Test Schedule

The project schedule is to be agreed by the team; calendar dates are intentionally not fabricated.
- Phase 1 – Test preparation: Review SRS/repository, identify test data, confirm build and environment.
- Phase 2 – Test design: Finalize test cases and RTM mapping for FR-002–FR-018, NFR-019–NFR-024, and SR-001–SR-005.
- Phase 3 – Component and API testing: Execute backend pytest suite and targeted API tests; record baseline results.
- Phase 4 – Integration and system testing: Execute end-to-end workflows for Customer, Employee, and Admin roles.
- Phase 5 – Non-functional/security testing: Execute security, response-time, reliability, accessibility, and backup/recovery checks.
- Phase 6 – Regression and acceptance: Retest fixes, run the full suite, update RTM, and prepare the test summary/sign-off.

8. Test Deliverables

- Approved Software Test Plan.
- Test case catalogue and synthetic test data.
- Updated Requirements Traceability Matrix (requirement-to-test mapping and execution status).
- Automated test execution output (pytest) and API test evidence (Postman, where applicable).
- Defect reports with severity, reproduction steps, evidence, owner, and retest status.
- Security, performance, usability/accessibility, reliability, and backup/restore test evidence.
- Final Test Summary Report documenting coverage, pass/fail/blocked counts, open defects, known gaps, and recommendation.

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Kruthik Nandan	Own the test plan, test strategy, RTM, execution coordination, defect triage, and test summary.
Test Engineer	Himesh Raghav	Design and execute manual/API/UI tests,

		maintain test data, capture evidence, and retest defects.
Developer	Janhavi Ramakrishnan	Support environment setup, diagnose defects, implement fixes, and provide unit/integration test evidence.
Product Owner	Kruthik Nandan	Clarify acceptance criteria, prioritize defects, review residual risks, and approve release/test sign-off.

10. Risks and Mitigation

Risk	Mitigation
SRS/repository mismatch or incomplete implementation evidence	Treat SRS as expected behaviour; inspect current routes/tests; record gaps; do not mark unverified requirements as passed.
Financial data inconsistency during deposit, withdrawal, or transfer	Use disposable DB snapshots; assert account balances and all ledger rows before/after; prioritize atomicity and rollback tests.
Role/ownership checks only enforced in the UI	Test direct API calls with altered IDs and each role; verify server-side denial and absence of data leakage.

11. Assumptions & Dependencies

- The latest approved SRS and the current main-branch repository are the baselines for test design; changes require RTM/test-case review.
- The README describes Python 3.10+, Flask, SQLite, React 18, Vite, pytest, and a Postman collection. Exact installed versions must be taken from the repository dependency files.
- The repository README reports 67 pytest test cases passing. This is a repository-reported baseline, not an independently rerun result in this document; the team must execute `cd backend` then `pytest -v` and attach fresh output.
- The SRS contains FR-018 (profile settings), NFR-022 (audit logging), and NFR-023 (UI/accessibility). These are in scope even though they are not explicitly listed in the current README's implemented-issues summary; verify their current source files, routes, and tests before marking implementation complete.
- Issue #22 audit logging and #23 UI/accessibility should be treated as verification items

because the README summary does not list them among its implemented issues, although it describes selected audit events elsewhere.
- No external payment gateway, core banking integration, SMS provider, or physical hardware is assumed.
- Test data and backup/restore operations are isolated from any non-test database.

12. Suspension & Resumption Criteria

Suspend testing if:
- The build cannot start, database schema/seed setup is unstable, or the test environment is unavailable.
- A Critical defect risks corrupting account balances, transaction ledgers, transfer atomicity, credentials, or customer data isolation.
- More than 30% of planned cases are blocked by one environment/build issue.
- A backup/restore test risks modifying the only copy of a database.

Resume testing if:
- The blocking defect is fixed or an isolated safe workaround is approved.
- The test environment and disposable database are recreated and smoke-tested.
- Test data is reset to a known state and affected cases are identified for rerun.

13. Test Case Management & Traceability

Test cases use stable IDs in the format TC-FR-###, TC-NFR-###, and TC-SR-###, matching the requirement identifier. Each case records objective, preconditions, test data, steps, expected result, priority, execution status, and evidence. Execute positive, negative, boundary, and role-based variants where applicable.

RTM rule: Every FR-002–FR-018, NFR-019–NFR-024, and SR-001–SR-005 must map to at least one test case. Initial status is Not Run (N/R); update only after execution. A requirement is not considered verified merely because a related code module or test file exists.

Repository baseline: the README reports 67 pytest cases passing. Re-run the suite on the exact commit under test and retain terminal output. Link each automated test name or manual evidence to the relevant test case ID. Annex A provides the initial test catalogue; the execution log/RTM is maintained during testing.

14. Test Metrics & Reporting

Metrics collected:
- Requirement coverage = requirements with at least one mapped test / total in-scope requirements.
- Test execution progress = executed cases / planned cases; report Pass, Fail, Blocked, and Not Run separately.
- Pass rate = passed / executed cases (do not count blocked or not-run cases as passed).
- Defect counts by severity, age, module, and status; reopen rate; and fix verification status.

- Security invariant results: unauthorized access denied, CSRF enforcement, password hashing, ledger consistency, and backup integrity.
- Performance observations for defined common operations, recording environment, sample size, median and percentile latency; compare to SRS targets/configuration.

Reports:
- Test execution log after each run; defect summary during triage; and final Test Summary Report at exit.
- Every report identifies repository commit, environment, test data version, date, and evidence location.

15. Approvals

Role	Name	Signature / Date
QA Lead	Kruthik Nandan	Pending
Development Lead	Janhavi Ramakrishnan	Pending
Product Owner	Himesh Raghav	Pending

Annex A – Initial Test Case Catalogue

Test Case ID	Requirement ID	Test Scenario and Steps	Preconditions / Test Data	Expected Result	Priority
TC-FR-002-01	FR-002	Valid login creates authenticated session Steps: Submit correct username/password.	Valid seeded test user; clean session	Login succeeds; protected page/API is accessible; session cookie is HttpOnly and signed.	High
TC-FR-002-02	FR-002	Invalid login and lockout boundary Steps: Try wrong password five times, then try correct password during lockout.	Known username; reset login-attempt state	Invalid attempts are rejected; fifth failure triggers 15-minute lockout; correct password is blocked during lockout.	High
TC-FR-003-01	FR-003 / SR-003	Role permissions and customer isolation Steps: Call protected endpoints as each role; substitute another customer's account/loan IDs.	Customer A, Customer B, Employee, Admin accounts	Only permitted role actions succeed; cross-customer records and restricted operations are denied server-side.	High
TC-FR-004-01	FR-004	Account number, balance, and status invariants Steps: Inspect account details; attempt closure with non-zero balance; verify paise values.	Test accounts in active/frozen/closed states	Account number is unique 12-digit value; balances are consistent; closure invariant and status rules are enforced.	High
TC-FR-005-01	FR-005	Deposit valid and invalid amounts Steps: Deposit positive amount; repeat with zero, negative, malformed, and frozen/closed account.	Active account with known balance	Valid deposit updates balance and ledger together; invalid amount/status is rejected with no partial update.	High
TC-FR-006-01	FR-006	Withdrawal boundaries and overdraft prevention Steps: Withdraw less than, equal to, and greater than balance; test zero/negative and frozen/closed account.	Active account with known balance	Valid withdrawal is recorded; insufficient funds and invalid amounts are rejected; balance never becomes negative.	High
TC-FR-007-01	FR-007	Transaction ledger completeness and uniqueness Steps: Perform deposit and withdrawal; inspect ledger reference, type, amount, timestamp, resulting balance.	Disposable DB and active account	Each successful operation has a unique reference and correct persistent ledger data; failed operations create no misleading success record.	High
TC-FR-008-01	FR-008	Valid internal transfer Steps: Transfer positive amount to valid destination account number; inspect both accounts and ledger.	Two active accounts owned/authorized for test	Source is debited, destination credited, and transfer_out/transfer_in records correspond to the same transfer.	High
TC-FR-008-02	FR-008	Reject invalid transfer variants Steps: Attempt self-transfer, unknown destination, non-owned source, inactive destination, and insufficient-funds transfer.	Active source; invalid, self, frozen, closed, or unauthorized destination	Every invalid transfer is rejected; neither account balance nor ledger is partially changed.	High
TC-FR-009-01	FR-009	Atomic transfer rollback on injected failure Steps: Cause failure after one transfer write but before all related writes complete.	Disposable DB; controlled failure point/test seam	Transaction rolls back; source/destination balances, ledger rows, and transfer record remain mutually consistent.	High
TC-FR-010-01	FR-010	Loan application validation	Customer account; supported loan types	Valid application is stored with unique reference and initial status;	Medium

Test Case ID	Requirement ID	Test Scenario and Steps	Preconditions / Test Data	Expected Result	Priority
		Steps: Submit valid personal/home/education/vehicle/business applications and invalid/missing fields.		invalid input is rejected and not persisted.	
TC-FR-011-01	FR-011 / SR-003	Loan review authorization and filtering Steps: Retrieve review queue as employee and customer; filter by status; attempt cross-customer detail access.	Applications in multiple statuses; Customer and authorized employee	Authorized staff can review/filter; customer or unauthorized role cannot access staff-only review functions or another user's data.	High
TC-FR-012-01	FR-012	Loan approval and rejection decision rules Steps: Approve one application; reject another with and without reason; inspect status and notification.	Pending loan; authorized reviewer	Authorized decision updates status and is recorded; rejection without mandatory reason is rejected; customer can see outcome.	Medium
TC-FR-013-01	FR-013	Chronological transaction history Steps: Open history and compare records against database ledger.	Account with deposits, withdrawals, transfers	History is chronological and displays correct reference, type, amount, and resulting balance; no other customer's data appears.	Medium
TC-FR-014-01	FR-014	Transaction filters, date boundaries, and pagination Steps: Search; filter by type; apply inclusive start/end dates; sort; navigate pages.	History containing multiple types and dates	Results match all selected filters and sort order; date endpoints are inclusive; no duplicate or missing rows across pages.	Medium
TC-FR-015-01	FR-015	CSV statement correctness and formula injection Steps: Export CSV; compare rows/headers; include text values beginning with =, +, -, or @ where accepted.	Account with representative transaction rows	CSV is complete and accurate; dangerous spreadsheet formulas are escaped; export is limited to authorized account data.	High
TC-FR-016-01	FR-016	Transaction and loan notifications Steps: Trigger deposit, withdrawal, transfer, loan approval/rejection; inspect unread count and mark-read.	Customer account; notification events available	Expected notification is created with correct event details; unread count changes and mark-read persists; delivery timing is measured against SRS SLA.	Medium
TC-FR-017-01	FR-017	Beneficiary lifecycle and transfer limits Steps: Add, view, update/manage, and delete beneficiary; attempt invalid beneficiary and transfer above configured limit.	Customer account; eligible/ineligible destination accounts	Eligible beneficiary changes persist; invalid destination and over-limit transfer are rejected; deleted beneficiary is no longer selectable.	Medium
TC-FR-018-01	FR-018	Profile update and password change Steps: Update permitted profile fields; try invalid email; change password with correct and incorrect current password; test password reuse.	Authenticated customer; current password known	Valid changes persist; invalid fields/current password and prohibited reuse are rejected; new password works and old password fails.	Medium
TC-NFR-019-01	NFR-019 / SR-001–004	Security baseline and sensitive response checks Steps: Inspect password storage/response, session cookie, CSRF checks, security headers, and cache headers.	All roles; browser/API inspection tools	Passwords are bcrypt-hashed; session/CSRF protections and configured security/cache headers are present; sensitive responses are not improperly cached.	High
TC-NFR-020-01	NFR-020	Response-time measurement Steps: Measure common login, balance,	Stable test environment; representative seeded data	Timing headers are present as specified; results are recorded with median/percentiles and compared	Medium

Test Case ID	Requirement ID	Test Scenario and Steps	Preconditions / Test Data	Expected Result	Priority
		history, deposit, and transfer API operations over repeated runs; capture timing headers.		with the SRS sub-second target under stated test conditions.	
TC-NFR-021-01	NFR-021	Health/readiness and database resilience Steps: Call liveness/readiness endpoints; simulate unavailable/busy DB where safe; interrupt request transaction.	Running API and disposable SQLite DB	Health endpoints report expected status; busy-timeout and teardown handling do not leave partial/uncommitted financial updates.	High
TC-NFR-022-01	NFR-022 / SR-005	Audit event coverage and access control Steps: Trigger login, account lifecycle, deposit, withdrawal, transfer success/rollback, loan/admin events; inspect authorized audit view/export.	Admin and non-admin accounts; test events	Required events capture actor/action/time/details; non-admin cannot read/export/alter logs; audit data is not exposed through customer endpoints.	High
TC-NFR-023-01	NFR-023	Responsive UI, labels, keyboard, and feedback Steps: Navigate key workflows by keyboard; inspect labels/focus, resize viewport, check reduced-motion and success/error feedback.	Desktop and narrow viewport; browser accessibility tools	Controls remain usable and labelled; focus is visible; layout adapts; feedback is understandable; reduced-motion preference is respected where implemented.	Medium
TC-NFR-024-01	NFR-024	Backup integrity and recovery Steps: Create backup; verify SHA-256 and SQLite integrity; restore to isolated target; test corrupt checksum and unauthorized restore.	Disposable DB copy; authorized admin	Valid backup verifies and restores; pre-restore safety snapshot is created; corrupt backup/unauthorized restore is rejected; original DB remains safe.	High
TC-SR-001	SR-001	Session and password protection Steps: Register/login; inspect stored password representation and session cookie; logout and retry protected access.	Test account; database inspection access	Password is bcrypt-hashed; signed HttpOnly session is used; logout invalidates authenticated access.	High
TC-SR-002	SR-002	CSRF enforcement Steps: Submit state-changing requests with missing, wrong, and correct X-CSRF-Token.	Authenticated session and valid CSRF token	Missing/invalid token requests are rejected; valid token requests proceed subject to authorization and validation.	High
TC-SR-003	SR-003	Server-side RBAC and ownership checks Steps: Attempt role escalation and cross-customer access directly through API URLs/payloads.	All three roles and two customer identities	Unauthorized actions and record access are denied regardless of frontend visibility or client-supplied IDs.	High
TC-SR-004	SR-004	Security and cache headers Steps: Inspect response headers for the project's required security/cache controls.	Representative public, authenticated, and sensitive endpoints	Required headers match project security configuration on applicable responses; sensitive pages/data have appropriate cache protection.	Medium
TC-SR-005	SR-005	Auditability and tamper resistance Steps: Generate supported events; attempt unauthorized read, export, update, and delete of audit records.	Admin and non-admin identities	Supported events are traceable; only authorized admin access is permitted; unauthorized tampering is blocked and recorded where specified.	High

Annex B – Requirements Traceability Matrix

Requirement ID	Requirement	Mapped Test Case(s)
FR-002	Authentication & Login	TC-FR-002-01, TC-FR-002-02
FR-003	Role-Based Access Control	TC-FR-003-01
FR-004	Account & Balance Management	TC-FR-004-01
FR-005	Deposit Money	TC-FR-005-01
FR-006	Withdraw Money	TC-FR-006-01
FR-007	Transaction Validation & Recording	TC-FR-007-01
FR-008	Fund Transfer Between Accounts	TC-FR-008-01, TC-FR-008-02
FR-009	Atomic Transfer & Rollback	TC-FR-009-01
FR-010	Loan Application	TC-FR-010-01
FR-011	Loan Application Review	TC-FR-011-01
FR-012	Loan Approval / Rejection & Status	TC-FR-012-01
FR-013	Transaction History	TC-FR-013-01
FR-014	Transaction Filtering & Pagination	TC-FR-014-01
FR-015	Transaction Statement / Export	TC-FR-015-01
FR-016	Transaction & Loan Notifications	TC-FR-016-01
FR-017	Beneficiary Management	TC-FR-017-01
FR-018	Profile and Account Settings	TC-FR-018-01
NFR-019	Data Security and Role Access	TC-NFR-019-01
NFR-020	System Response Time	TC-NFR-020-01
NFR-021	System Availability and Reliability	TC-NFR-021-01
NFR-022	Audit Logging	TC-NFR-022-01
NFR-023	User Interface and Accessibility	TC-NFR-023-01
NFR-024	Backup and Recovery	TC-NFR-024-01
SR-001	Signed sessions and bcrypt	TC-SR-001
SR-002	CSRF validation	TC-SR-002
SR-003	RBAC and customer data isolation	TC-SR-003
SR-004	Security response/cache headers	TC-SR-004
SR-005	Audit event traceability	TC-SR-005